

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

Islamic University of Technology

May 16, 2026



Table of Contents

V model

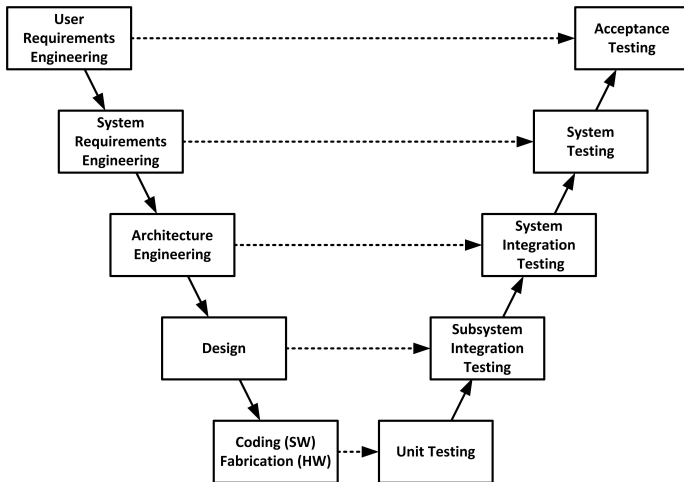


Figure 1: V Model

Limitations of Testing

- **Testing shows the presence of errors, not their absence**
 - Exhaustive testing is only feasible for trivial systems
- **Test coverage is inherently limited**
 - Selection of test cases is subjective
 - Injected faults may not represent real-world failures
- **Difficult to test unexpected scenarios**
 - Rare edge cases are often missed
 - Unknown conditions are hard to anticipate
- **Testing is costly**
 - Requires time, effort, and resources

Complementing Testing: Formal Verification

Testing a `sort()` Function

- Typical test cases:
 - `sort({3,2,5}) = {2,3,5}`
 - `sort({}) = {}`
 - `sort({17}) = {17}`

Often Missed Cases

- Duplicate elements:
 - `sort({2,1,2}) = {1,2,2}`
- Invalid input:
 - `sort(null) → exception`

Key Property (Formal Verification)

- `isPermutation(sort(a), a)`

Insight

Testing checks examples,

Formal verification proves properties for **all possible inputs**.

Formal Verification as Theorem Proving

Theorem (Correctness of `sort()`)

For any non-null integer array a , the function `sort(a)` returns an array that:

- is sorted with respect to $\leq$
- is a permutation of the input array a

How is this different from Mathematics?

- Formalize the expected properties using a logical language
- Use (semi-)automated tools to assist in proving correctness

Formal Methods

- Mathematical techniques for specifying and verifying software systems
- Examples:
 - Theorem proving
 - Model checking
 - Static analysis

Definition

- Formal methods are rigorous techniques and tools for the development and analysis of computational systems (hardware and software).

Key Characteristics

- Based on mathematics and symbolic logic
- Provide precise and unambiguous specifications

Formal methods aim to ensure correctness through mathematical reasoning.

Where are Formal Methods Used?

- Applied at various stages of the development lifecycle
- Specification and system design
- Verification of correctness
- Reverse engineering to model and analyze existing systems

Why are they important?

- Improve reliability and safety
- Detect errors early in development
- Provide strong guarantees of system behavior

Two Key Artifacts

- **System Requirements**
 - What the system is supposed to do
- **System Implementation**
 - How the system is actually built

Goal

Ensure that the implementation correctly satisfies the requirements.

How Formal Methods Work

Key Components

- Formal specification of requirements
- Formal execution model of implementation

Verification Process

- Use automated or semi-automated tools
- Mechanically verify that:
 - Implementation satisfies the specification

Key Insight

Formal verification bridges the gap between
what we want and **what we build**.

Definition

- A specification describes **what the system should do**
- It defines expected behavior in a precise and unambiguous way

Types of Properties

- **Safety Properties**

- Something bad will never happen

- **Liveness Properties**

- Something good will eventually happen

- **Non-functional Properties**

- Performance, memory usage, usability, etc.

Beyond Simple Properties

- Specifications can describe complete system behavior

Examples

- **Equivalence Checking**
 - Two systems behave identically
- **Refinement**
 - A concrete system correctly implements an abstract model
- **Data Consistency**
 - System maintains valid and consistent data

Definition

A formal specification is the expression, in a formal language and at some level of abstraction, of a collection of properties that a system should satisfy.

Key Aspects

- **Formal Language**

- Syntax can be mechanically processed and checked
- Semantics is defined unambiguously using mathematics

Key Elements of Formal Specifications

- **Abstraction**

- Describes the system above the level of source code
- Multiple levels of abstraction are possible

- **Properties**

- Expressed using formal logic
- Have precise and well-defined meaning

- **Satisfaction**

- Ideally verified mechanically
- Uses automated deduction and/or model checking techniques

Key Insight

Formal specifications enable precise reasoning about system correctness.

Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`